

Adult Income Prediction — Day 2 Write-Up

Preprocessing Choices

The Adult dataset has six numeric features (`age` , `fnlwgt` , `education-num` , `capital-gain` , `capital-loss` , `hours-per-week`) and eight categorical features (`workclass` , `education` , `marital-status` , `occupation` , `relationship` , `race` , `sex` , `native-country`). I converted `?` to NaN the same way as Day 1.

I used a `ColumnTransformer` with separate pipelines for each type. For numeric columns I applied median imputation and then `StandardScaler` . I chose median over mean because `capital-gain` and `capital-loss` are skewed and a few large values would pull the mean up. Scaling helps logistic regression compare coefficients on a similar scale. Trees do not need scaling, but one shared preprocessor keeps both models consistent and avoids leakage.

For categoricals I used `SimpleImputer(strategy="constant", fill_value="Missing")` and `OneHotEncoder(handle_unknown="ignore")` . Missing values become their own category instead of being dropped. One-hot encoding fits here because categories like `workclass` or `education` have no natural order. I did not use label encoding since that would treat categories as if they were ranked.

Everything runs inside sklearn `Pipeline` objects. The models were fit on the training set only; the Day 1 hold-out test set was used only for final evaluation.

I kept both `education` and `education-num` for now even though they overlap. I plan to test dropping one of them in Day 3.

Model Comparison

Model	Accuracy	Precision	Recall	F1	ROC AUC	PR AUC
Majority Baseline (Day 1)	0.7607	0.0000	0.0000	0.0000	0.5000	0.2393
Education Rule (Day 1)	0.7530	0.4844	0.4970	0.4906	0.6653	0.3611
Logistic Regression	0.8543	0.7427	0.5988	0.6630	0.9057	0.7670
Decision Tree	0.8186	0.6185	0.6317	0.6251	0.7546	0.4789

Both models beat the Day 1 baselines on F1. Logistic Regression did best overall (F1 = 0.6630 vs 0.6251 for the tree), with higher ROC AUC (0.9057 vs 0.7546) and PR AUC (0.7670 vs 0.4789).

Error analysis and F1

On the test set, Logistic Regression had 485 false positives and 938 false negatives. False negatives were more common, so recall (0.5988) came in below precision (0.7427). Since F1 balances both types of error, I want to improve recall in Day 3 without losing too much precision.

The Decision Tree overfit badly: train accuracy was about 99.99% but test accuracy was 0.8186, with depth 68 and 5,668 leaves. The first splits on `marital-status`, `capital-gain`, and `education-num` make sense, but the full tree is too complex.

Model Selection for Day 3

I will continue with **Logistic Regression** in Day 3. It had the highest F1 and AUC on the test set, train and test accuracy were both around 0.85, and the coefficients are easy to read. Higher `capital-gain` , `education-num` , and `Married-civ-spouse` all pushed predictions toward above 50K USD.

The **Decision Tree** was useful for spotting nonlinear patterns, but the large train/test gap means I would need `max_depth` or `min_samples_leaf` tuning before relying on it.

Plans for Day 3

1. Split training data into train/validation sets for hyperparameter tuning.
2. Try `class_weight="balanced"` to improve recall on the minority class.
3. Drop either `education` or `education-num` to reduce redundancy.
4. If I revisit trees, tune `max_depth` and `min_samples_leaf` .
5. Reuse `log_reg_pipeline.joblib` as a starting point.